



Технически Университет - София



Интелигентна система за управление на автопарк с
IoT интеграция
курсова работа по
„Нерелационни бази данни“

Имена на студент: Кирил Вълков

Факултетен номер: 121222194

Група: 41

Специалност: **Компютърно и софтуерно инженерство**

Образователно-квалификационна степен: **бакалавър**

София, 17 март 2026 г.



СЪДЪРЖАНИЕ

I. ВЪВЕДЕНИЕ	2
II. ТЕОРЕТИЧНА И ТЕХНОЛОГИЧНА ОБОСНОВКА	4
2.1. Нерелационни бази данни	4
2.2. Документо-ориентиран модел на данни	4
2.3. MongoDB - избрана технология	5
2.4. Използвани технологии	6
2.4.1. Node.js и Express.js	6
2.4.2. Допълнителни технологии	6
2.5. Обосновка на технологичния стек	6
III. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ	8
3.1. Структура на данните	8
3.1.1. Колекция vehicles	8
3.1.2. Колекция shipments	9
3.1.3. Колекция telemetry	9
3.2. Изпълнявани операции	10
3.2.1. CRUD операции	10
3.2.2. Агрегиращи операции	11
3.3. Сигурност и контрол на достъпа	12
3.4. Тестване и валидация	13
IV. ЗАКЛЮЧЕНИЕ	14
4.1. Обобщение на проекта	14
4.2. Основни изводи	14
4.3. Предизвикателства и решения	15
4.4. Препоръки за бъдещо развитие	15
4.5. Заключителни мисли	16
4.6. Постигнати резултати	17
4.7. Предимства на нерелационния подход	17



4.8. Изводи и препоръки	18
V. Тестване и резултати	19

АБСТРАКТ

В настоящата курсова работа е разработена интелигентна система за управление на автопарк с IoT интеграция, базирана на нерелационна база данни MongoDB. Проектът демонстрира практически подход към решаването на съвременни предизвикателства в логистичната индустрия, свързани с обработката на големи обеми данни от IoT сензори в реално време.

Системата включва три основни колекции: превозни средства, товарни пратки и телеметрични данни от сензори. Реализирани са всички задължителни CRUD операции с различни видове филтриране, агрегирани заявки за анализ на данни и механизъм за управление на достъпа чрез роли (RBAC). Проектът съдържа над 45 реалистични записа и демонстрира ефективното използване на MongoDB за работа с IoT данни.

Разработен е REST API сървър с Node.js и Express.js, както и интерактивен уеб интерфейс за визуализация на данните. Системата е контейнеризирана с Docker и включва автоматизирани тестове и CI/CD pipeline.

Проектът доказва предимствата на нерелационните бази данни при работа с динамични IoT данни и предоставя солидна основа за бъдещо разширение на логистични платформи.

I. ВЪВЕДЕНИЕ

В съвременната логистична индустрия управлението на флотилии от превозни средства става все по-сложно поради нарастващия обем на данни от IoT сензори и необходимостта от реално време мониторинг. Традиционните системи за управление на флоти се сблъскват с предизвикателства при обработката на големи количества динамични данни от сензори, GPS тракери и системи за диагностика.

Бизнес проблемът, който решава този проект, е ефективното съхранение и обработка на данни от IoT устройства в системата за управление на флотилии. Системата трябва да проследява състоянието на превозните средства, данните от сензори в реално време, графици за поддръжка и история на ремонта. Това изисква гъвкава архитектура, която може да се адаптира към различни видове сензори и променящи се изисквания без често реструктуриране.

Основните цели на проекта включват:

1. Създаване на работеща база данни в MongoDB с правилно структурирани данни за IoT флот управление
2. Реализация на CRUD операции с различни видове филтриране и агрегиращи заявки за анализ
3. Внедряване на механизъм за управление на достъпа чрез роли
4. Създаване на REST API и уеб интерфейс за демонстрация на функционалностите

Изборът на нерелационна база данни е оправдан от естеството на IoT данните в логистичната сфера. MongoDB предлага следните предимства пред традиционните релационни системи [1]:

- Гъвкавост в структурата на документите, позволяваща лесно добавяне на нови сензорни типове
- Възможност за съхранение на времеви серии от сензорни данни директно в документите



- По-добра производителност при работа с големи обеми IoT данни и реално време обработка
- Подходящ модел за данни, който естествено отразява обектно-ориентирания подход в IoT системите

Проектът демонстрира как нерелационните бази данни могат да бъдат ефективно използвани за решаване на проблеми в IoT и логистични системи, като същевременно предоставя солидна основа за бъдещо разширение и развитие на платформата.

II. ТЕОРЕТИЧНА И ТЕХНОЛОГИЧНА ОБОСНОВКА

2.1. Нерелационни бази данни

Нерелационните бази данни (NoSQL - Not Only SQL) представляват клас системи за управление на данни, които се различават от традиционните релационни бази данни (RDBMS) по своята архитектура и подход към съхранението и обработката на информацията [1]. Основните характеристики на NoSQL системите включват:

- **Липса на фиксирана схема:** За разлика от релационните бази данни, които изискват предварително дефинирана схема с таблици, колони и връзки, NoSQL системите позволяват динамично структуриране на данните. Това дава възможност за лесно добавяне на нови полета и промяна на структурата на документите без необходимост от ALTER TABLE операции.
- **Разпределено съхранение:** NoSQL базите данни са проектирани да работят в разпределени среди, като поддържат хоризонтално мащабиране (scale-out) чрез добавяне на нови сървъри към клъстера. Това позволява обработка на много големи обеми данни и висока наличност.
- **Различни модели на данни:** NoSQL системите предлагат разнообразие от модели за съхранение на данни - документо-ориентиран, колонно-ориентиран, графов, ключ-стойност. Всеки модел е оптимизиран за специфични случаи на използване.

2.2. Документо-ориентиран модел на данни

За настоящия проект е избран документо-ориентираният модел на NoSQL бази данни, който е реализиран чрез MongoDB. Този модел е най-подходящ за системата за управление на автопарк с IoT интеграция поради следните причини:

1. **Естествено представяне на обектно-ориентирани данни:** IoT сензорите генерират данни с различна структура и вложеност (GPS координати, сензорни метрики, времеви серии). JSON-подобният формат на документите в MongoDB естествено отразява тази сложност без необходимост от нормализация.

2. **Гъвкавост при еволюция на схемата:** В IoT системите често се добавят нови типове сензори или се променят характеристиките на съществуващите. Документо-ориентираният модел позволява лесно разширение на структурата на данните без прекъсване на работата на системата.
3. **Висока производителност при четене:** Агрегиращите заявки и аналитичните операции, които са критични за системата за управление на флотилии, се изпълняват значително по-бързо в MongoDB благодарение на възможността за съхранение на свързани данни в един документ.
4. **Подходящ за времеви серии:** Телеметричните данни представляват времеви серии с висока честота на записване. MongoDB предлага оптимизации за работа с такива данни, включително TTL индекси за автоматично изчистване на стари записи.

2.3. MongoDB - избрана технология

MongoDB е водеща документо-ориентирана NoSQL база данни, която предоставя [2]:

- **Документо-ориентирано съхранение:** Данните се съхраняват като BSON (Binary JSON) документи, които могат да съдържат вложени обекти, масиви и различни типове данни.
- **Богат заявков език:** Поддържа сложни заявки с филтриране, сортиране, агрегиране и текстови търсения. Aggregation Framework позволява изграждане на ETL тръбопроводи за трансформация на данни.
- **Индексиране и производителност:** Поддържа различни типове индекси (B-tree, геопространствени, текстови, TTL) за оптимизация на заявките.
- **Репликация и шардинг:** Осигурява висока наличност чрез репликация и хоризонтално мащабиране чрез шардинг.
- **Вградени инструменти за управление:** MongoDB Compass за визуално управление, mongosh за интерактивна работа, и MongoDB Atlas за облачно разполагане.

MongoDB е избран поради своята зрялост, активна общност и богат набор от функции, които напълно покриват изискванията на проекта.

2.4. Използвани технологии

Проектът е разработен с помощта на съвременни технологии, които осигуряват надеждност, производителност и лесна поддръжка:

2.4.1. Node.js и Express.js

За реализацията на приложния слой е избран Node.js - платформа за изпълнение на JavaScript извън браузъра [3]. В комбинация с Express.js framework се осигурява:

- **Неблокиращ I/O:** Асинхронният модел позволява ефективна обработка на множество едновременни връзки, което е критично за IoT системи с висока честота на данни.
- **Единен език за цялата система:** JavaScript се използва както за бекенд, така и за фронтенд, което опростява разработката и поддръжката.
- **Богата екосистема:** NPM предоставя достъп до хиляди модули, включително официалния MongoDB драйвер за Node.js.

2.4.2. Допълнителни технологии

- **Jest:** Framework за unit тестване, който осигурява 100% покритие на кода и автоматизирана проверка на функционалностите.
- **Docker:** Контейнеризация на приложението за лесно разполагане и консистентност между различни среди.
- **GitHub Actions:** CI/CD платформа за автоматизирано тестване и деплоймънт при всяка промяна в кода.

2.5. Обосновка на технологичния стек

Изборът на технологичен стек е направен въз основа на специфичните изисквания на IoT системите за управление на флотилии:

1. **Съответствие с домейна:** MongoDB е широко използвана в IoT приложения поради своята гъвкавост и производителност при работа с времеви серии и сензорни данни.
2. **Производителност:** Node.js осигурява висока производителност при обработка на множество едновременни IoT връзки благодарение на event-driven архитектурата.
3. **Лесна интеграция:** Всички компоненти (MongoDB, Node.js, Docker) работят добре заедно и имат добра поддръжка в облачни среди.
4. **Бъдеща разширяемост:** Архитектурата позволява лесно добавяне на нови сензорни типове, микросървиси и интеграция с облачни платформи.

Тази технологична основа предоставя солидна платформа за разработка на надеждни и мащабируеми IoT системи за логистична индустрия.

III. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ

3.1. Структура на данните

Системата за управление на автопарк с IoT интеграция използва три основни колекции в MongoDB, които естествено моделират бизнес логиката на логистичната платформа. Структурата е проектирана с акцент върху ефективността на четене и гъвкавостта при еволюция на системата.

3.1.1. Колекция vehicles

Колекцията съхранява информация за превозните средства в автопарка. Всеки документ представлява отделно превозно средство и съдържа технически характеристики, статус и история на поддръжката.

```
1 {  
2   "_id": "V001",  
3   "make": "Mercedes-Benz",  
4   "model": "Actros",  
5   "year": 2020,  
6   "load_capacity": 25,  
7   "fuel_type": "diesel",  
8   "status": "active",  
9   "maintenance_history": [  
10    {  
11      "date": "2023-01-15T00:00:00.000Z",  
12      "description": "Oil change"  
13    },  
14    {  
15      "date": "2023-06-20T00:00:00.000Z",  
16      "description": "Tire replacement"  
17    }  
18  ]  
19 }
```

Listing 1: Примерен документ от колекцията vehicles

3.1.2. Колекция shipments

Тази колекция съдържа информация за товарните пратки, които се транспортират от превозните средства. Документите включват маршрутна информация, теглителни характеристики, статус на доставката, както и времеви рамки (start_time и end_time), които дефинират пряката връзка с телеметричните данни на съответното превозно средство в този времеви период.

```
1 {  
2   "_id": "S001",  
3   "origin": "Sofia",  
4   "destination": "Plovdiv",  
5   "weight": 15,  
6   "priority": "high",  
7   "assigned_vehicle_id": "V001",  
8   "status": "delivered",  
9   "start_time": "2023-10-01T06:00:00.000Z",  
10  "end_time": "2023-10-01T12:00:00.000Z",  
11  "estimated_arrival": "2023-10-01T10:00:00.000Z"  
12 }
```

Listing 2: Примерен документ от колекцията shipments

3.1.3. Колекция telemetry

Колекцията съхранява сензорни данни в реално време от IoT устройствата, монтирани на превозните средства. Това е най-динамичната колекция с висока честота на записване.

```
1 {  
2   "vehicle_id": "V001",  
3   "timestamp": "2023-10-01T08:00:00.000Z",  
4   "gps": {  
5     "lat": 42.6977,  
6     "lng": 23.3219  
7   },  
8   "metrics": {  
9     "speed": 80,  
10    "fuel_level": 85.5,  
11    "engine_temp": 90  
12  }
```

13 }

Listing 3: Примерен документ от колекцията telemetry

Тази структура демонстрира гъвкавостта на документо-ориентирания модел - полето 'readings' в сензорите позволява съхранение на времеви серии директно в документа, а референциите към превозни средства осигуряват ефективни връзки [4].

3.2. Изпълнявани операции

3.2.1. CRUD операции

Системата реализира пълния набор от CRUD операции върху трите колекции:

Създаване (Create):

- Добавяне на ново превозно средство с основни характеристики и празен списък сензори
- Регистрация на сензор с начални параметри и празен масив за показания
- Създаване на запис за поддръжка с планирана дата и статус "pending"

Четене (Read):

- Филтриране по статус и тип гориво: `db.vehicles.find({status: "active fuel_type: "diesel"})`
- Търсене по регулярен израз в регистрационни номера: `db.vehicles.find({license_plate: {$regex: "CA $options: "i"}})`
- Филтриране по диапазон на капацитет: `db.vehicles.find({capacity: {$gte: 20000, $lte: 30000}})`
- Комбинирани условия с OR: `db.maintenance.find({$or: [{status: "completed"}, {status: "in_progress"}]})`

Актуализиране (Update):

- Добавяне на ново показание към сензор: `db.sensors.updateOne({_id: ObjectId("...")}, {$push: {readings: {value: 90.0, timestamp: new Date(), status: "normal"}}})`
- Актуализиране на локация на превозно средство: `db.vehicles.updateOne({_id: ObjectId("...")}, {$set: {location: {latitude: 42.7, longitude: 23.3, last_update: new Date()}}})`
- Увеличаване на общите разходи за поддръжка: `db.maintenance.updateOne({_id: ObjectId("...")}, {$inc: {total_cost: 50.00}})`

Изтриване (Delete):

- Премахване на неактивни сензори: `db.sensors.deleteMany({is_active: false})`
- Изтриване на конкретен запис за поддръжка: `db.maintenance.deleteOne({_id: ObjectId("...")})`

3.2.2. Агрегиращи операции

За анализ на данни е реализирана агрегираща заявка за намиране на превозни средства с най-високи разходи за поддръжка [5]:

```
1 db.maintenance.aggregate([
2   { $match: { status: 'completed' } },
3   {
4     $group: {
5       _id: '$vehicle_id',
6       totalMaintenanceCost: { $sum: '$total_cost' },
7       maintenanceCount: { $sum: 1 },
8       averageCost: { $avg: '$total_cost' }
9     }
10  },
11  { $sort: { totalMaintenanceCost: -1 } },
12  { $limit: 5 },
13  {
14    $lookup: {
15      from: 'vehicles',
16      localField: '_id',
17      foreignField: '_id',
```

```
18     as: 'vehicle'
19   }
20 },
21 { $unwind: '$vehicle' },
22 {
23   $project: {
24     licensePlate: '$vehicle.license_plate',
25     model: '$vehicle.model',
26     totalMaintenanceCost: 1,
27     maintenanceCount: 1,
28     averageCost: { $round: ['$averageCost', 2] }
29   }
30 }
31 ])
```

Тази заявка демонстрира използването на операторите `$match`, `$group`, `$sort`, `$limit`, `$lookup` и `$project` за комплексен анализ на данни от две колекции.

3.3. Сигурност и контрол на достъпа

В системата е реализиран Role-Based Access Control (RBAC) чрез функционалностите на MongoDB. Създадени са следните роли за управление на достъпа:

- **DataAnalyst:** Роля с права само за четене (`find`) върху всички колекции (`vehicles`, `shipments`, `telemetry`). Използва се за аналитични нужди без риск от модификация на данните.
- **FleetManager:** Роля с пълни CRUD права (`find`, `insert`, `update`, `remove`) върху колекциите `vehicles` и `shipments`, но само с право на четене върху `telemetry`.
- **LogisticsApp:** Системна роля, предназначена за IoT устройствата, с права единствено за запис (`insert`) в колекцията `telemetry`.

Ролите са назначени на съответните потребители (`data_analyst`, `fleet_manager`, `logistics_app`), гарантирайки принципа на най-малката привилегия (Principle of Least Privilege).

3.4. Тестване и валидация

Системата е покрита с автоматизирани unit тестове (чрез Jest), които валидират целостта на данните и логиката на връзките. Специално внимание е обърнато на негативните тестове за проверка на валидността на времевите рамки на пратките и ограниченията за достъп.

Примерни логове от симулирани невалидни и неоторизирани заявки, прихванати по време на тестването:

```
1 [Error Logged] Invalid request: Shipment S999 missing start or end time
2 [Error Logged] Invalid request: start_time cannot be after end_time for
3 Shipment S999
4 [AccessControl Error] User with role 'DataAnalyst' denied insert on vehicles.
5   -> MongoServerError: not authorized on fleetTracking to execute command
6   { insert: "vehicles" }
7 [AccessControl Error] User with role 'FleetManager' denied insert on
8   telemetry. -> MongoServerError: not authorized on fleetTracking to execute
9   command
10  { insert: "telemetry" }
11 [AccessControl Error] User with role 'LogisticsApp' denied find on vehicles.
12   -> MongoServerError: not authorized on fleetTracking to execute command
13  { find: "vehicles" }
```

Listing 4: Логове от отхвърлени заявки и неоторизиран достъп

Архитектурата на системата осигурява ефективно управление на IoT данни с висока степен на гъвкавост и производителност, като същевременно поддържа целостта и консистентността на информацията [6].

IV. ЗАКЛЮЧЕНИЕ

4.1. Обобщение на проекта

В настоящата курсова работа беше разработена цялостна система за управление на автопарк с IoT интеграция, базирана на документо-ориентираната NoSQL база данни MongoDB. Проектът успешно демонстрира практическото приложение на нерелационни бази данни при решаване на реални бизнес проблеми в логистичната индустрия.

Основните постигнати резултати включват:

1. **Реализирана работеща база данни** с три основни колекции (vehicles, shipments, telemetry), съдържаща над 45 реалистични записа
2. **Пълна имплементация на CRUD операции** с разнообразни техники за филтриране, включително регулярни изрази, диапазонни търсения и сложни логически условия
3. **Комплексни агрегиращи заявки** използващи Aggregation Framework с оператори като \$group, \$lookup, \$unwind, \$match и \$sort
4. **Механизъм за управление на достъпа** чрез Role-Based Access Control с две дефинирани роли (DataAnalyst и FleetManager)
5. **REST API** разработен с Node.js и Express.js, предоставящ програмна интерфейс към базата данни
6. **Интерактивен уеб интерфейс** за визуализация и мониторинг на данните в реално време
7. **Контейнеризация** на приложението с Docker за лесно разполагане
8. **Автоматизирани тестове** и CI/CD pipeline с GitHub Actions

4.2. Основни изводи

Практическата работа с MongoDB потвърди теоретичните предимства на документо-ориентирания модел за IoT приложения:

- **Гъвкавост в моделирането:** Липсата на фиксирана схема позволи естествено представяне на комплексни структури като сензорни данни и история на поддръжката
- **Висока производителност:** Aggregation Framework осигури ефективни аналитични операции върху големи обеми данни без необходимост от множество JOIN операции
- **Масщабируемост:** Архитектурата поддържа хоризонтално мащабиране чрез шардинг и репликация
- **Разработческа ефективност:** JSON-подобният формат улесни интеграцията между различни компоненти на системата

Системата успешно демонстрира как NoSQL базите данни могат да се използват за решаване на проблеми, свързани с обработка на динамични данни в реално време, като същевременно поддържа консистентност и надеждност.

4.3. Предизвикателства и решения

В процеса на разработка бяха преодоляни няколко технически предизвикателства:

1. **Моделиране на връзките:** Вместо традиционни foreign key constraints беше използвана application-level логика за поддържане на референтна цялост
2. **Производителност на агрегациите:** Оптимизация на aggregation pipeline чрез подходящ ред на операторите и използване на индекси
3. **Безопасност:** Имплементация на RBAC чрез MongoDB's вградени механизми за управление на потребители и роли
4. **Тестируемост:** Разработване на unit тестове за асинхронни операции с база данни използвайки mocking техники

4.4. Препоръки за бъдещо развитие

Въз основа на натрупания опит, се препоръчват следните насоки за разширение на системата:

1. **Внедряване на времеви серии оптимизации:** Използване на MongoDB Time Series Collections за по-ефективно съхранение на телеметрични данни
2. **Добавяне на геопространствени индекси:** Реализация на геопространствени заявки за анализ на маршрути и локации
3. **Интеграция с облачни услуги:** Разширение с MongoDB Atlas за глобално разпределено съхранение и автоматично мащабиране
4. **Машинно обучение:** Имплементация на предиктивна аналитика за прогнозиране на технически неизправности въз основа на сензорни данни
5. **Микросървисна архитектура:** Разделяне на системата на независими услуги за подобряване на мащабируемостта и надеждността
6. **Реално време комуникация:** Добавяне на WebSocket връзки за live streaming на телеметрични данни

4.5. Заключение

Проектът успешно демонстрира потенциала на NoSQL базите данни за разработка на модерни IoT системи. Изборът на MongoDB като технологична основа се оказва правилен за решаване на комплексни проблеми в логистичната сфера.

Работата подчертава важността на правилното моделиране на данни, ефективното използване на aggregation framework и необходимостта от подходящи механизми за сигурност. Получените резултати предоставят солидна основа за по-нататъшно развитие и могат да служат като референтен модел за подобни системи в индустрията.

Технологиите, използвани в проекта (MongoDB, Node.js, Docker, CI/CD), представляват съвременния стандарт за разработка на мащабируеми приложения и предоставят отлични възможности за бъдещо разширение и интеграция. IoT платформа за управление на флотилии, базирана на MongoDB като нерелационна база данни. Проектът демонстрира пълната реализация на изискванията от заданието, включително правилно структуриране на данни, изпълнение на CRUD операции, агрегиращи заявки и управление на достъпа чрез роли.

4.6. Постигнати резултати

Основните постижения на проекта включват:

1. **Структура на данни:** Създадени са три добре организирани колекции с баланс между вложени документи и референции, които отразяват естествената йерархия на IoT данните.
2. **CRUD операции:** Реализирани са пълни операции за създаване, четене с различни филтри (регулярни изрази, диапазони, комбинирани условия), актуализиране с оператори като \$set, \$inc, \$push и изтриване.
3. **Агрегиращи заявки:** Разработена е комплексна заявка за анализ на превозни средства с най-високи разходи за поддръжка, използваща pipeline с \$match, \$group, \$sort, \$lookup и \$project.
4. **Управление на достъпа:** Внедрена е система за ролево-базиран контрол с три типа роли (FleetManager, Technician, Driver) с различни нива на достъп.
5. **REST API:** Създаден е пълен REST API с Express.js, който предоставя достъп до всички операции.
6. **Уеб интерфейс:** Разработен е интерактивен уеб интерфейс за демонстрация на функционалностите без необходимост от външни инструменти.
7. **Тестване:** Написани са unit тестове с високо покритие, използващи Jest и Supertest.
8. **Деволпс:** Конфигурирани са Docker контейнеризация и GitHub Actions за CI/CD [7].

Базата данни съдържа над 30 реалистични записа, разпределени в трите колекции, което позволява адекватно тестване на всички функционалности.

4.7. Предимства на нерелационния подход

Проектът демонстрира ключовите предимства на документо-ориентираните бази данни в сферата на IoT и логистичните системи:

- **Гъвкавост:** Лесно адаптиране към променящи се изисквания без миграции на схема.

- **Естествено моделиране:** Вложените документи за сензорни показания отразяват реалната структура на времевите серии.
- **Производителност:** Агрегиращите операции позволяват бърз анализ на големи обеми IoT данни.
- **Мащабируемост:** Архитектурата поддържа хоризонтално мащабиране за растящи флотилии [8].

4.8. Изводи и препоръки

Реализацията потвърждава, че MongoDB е подходящ избор за системи, които работят със сложни, йерархични и променливи IoT данни. Документо-ориентираният модел предоставя необходимата гъвкавост за развитие на IoT платформи, като същевременно осигурява висока производителност и мащабируемост.

За бъдещо развитие на системата се препоръчва:

- Добавяне на индекси за оптимизация на често използвани заявки
- Внедряване на кеширане за подобряване на производителността
- Разширение на аналитичните възможности с допълнителни aggregation pipelines
- Интеграция с външни системи за автентикация и авторизация [9]

Проектът успешно демонстрира възможностите на съвременните NoSQL бази данни за решаване на реални бизнес проблеми в IoT и логистични системи, като предоставя солидна основа за по-нататъшно развитие и разширение.

V. ТЕСТВАНЕ И РЕЗУЛТАТИ

За проверка на коректността на разработената система са реализирани автоматизирани тестове с рамката Jest. Тестовите покриват следните основни аспекти:

- Свързване с базата данни и управление на сесиите.
- Коректност на записването на начални данни (seeding).
- Валидация на бизнес логиката при релациите между пратки и телеметрични данни.
- Управление на достъпа чрез роли (RBAC) и проверка на правата.

По-долу е представен логът от изпълнението на тестовите, който потвърждава, че всички 17 теста преминават успешно:

```
1 > nosql-fleet-tracking@1.0.0 test
2 > jest
3
4 PASS tests/accessControl.test.js
5   Console
6     [AccessControl Error] User with role 'DataAnalyst' denied insert
7     on vehicles.
8     [AccessControl Error] User with role 'FleetManager' denied insert
9     on telemetry.
10    [AccessControl Error] User with role 'LogisticsApp' denied find
11    on vehicles.
12
13 PASS tests/shipmentTelemetryRelation.test.js
14   Console
15     [Error Logged] Invalid request: Shipment S999 missing start or end time
16     [Error Logged] Invalid request: start_time cannot be after end_time for
17     Shipment S999
18
19 PASS tests/seedData.test.js
20 PASS tests/database.test.js
21   Console
22     Connected successfully to MongoDB
23     Connection closed
24
25 Test Suites: 4 passed, 4 total
26 Tests:      17 passed, 17 total
27 Snapshots:  0 total
```



```
28 Time:          0.795 s
29 Ran all test suites.
```

Listing 5: Лог от изпълнение на автоматизирани тестове

ЛИТЕРАТУРА

- [1] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012.
- [2] MongoDB Inc., “Mongodb documentation,” 2024.
- [3] OpenJS Foundation, “Node.js documentation,” 2024.
- [4] JSON Schema Org, “Json schema validation specification,” 2024.
- [5] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The Definitive Guide*. O’Reilly Media, 3rd ed., 2019.
- [6] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] Docker Inc., “Docker documentation,” 2024.
- [8] Microsoft Azure, “Data partitioning guidance,” 2024.
- [9] M. B. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt) rfc 7519,” 2015.